Computational Methods for Single-Cell RNA-Seq Analysis of *Arabidopsis thaliana*

Chapter 1 - WT vs pifq tutorial: data download, Cell Ranger processing, and Seurat analysis

SingleCell Pipeline

2026-07-15

Materials and Methods

Purpose of this document

This document rewrites Chapter 1 as a practical tutorial for the WT/pifq dataset. It starts by cloning the repository and then treats that clone as the project root for every later command. Raw reads, reference files, Cell Ranger outputs, Docker execution, and Chapter 1 results are all placed under that same Git working directory. The document stops at Chapter 1.

Working directory and repository

The first step is to clone the SingleCell repository. After entering the clone, `PROJECT_DIR="$(pwd)"` becomes the root used by the rest of the tutorial. This keeps each dataset self-contained: data, references, Cell Ranger output, marker files, and downstream results are stored inside the cloned repository folder, while Git tracks only the code and documentation.

bash

```
git clone https://github.com/cliford2001/SingleCell.git
cd SingleCell

PROJECT_DIR="$(pwd)"
mkdir -p data/raw references cellranger
export PATH="$PROJECT_DIR/tools/cellranger:$PATH"

git status
```

In this layout, `PROJECT_DIR` is the cloned `SingleCell` folder. The adapted Chapter 1 script is `workflow/capitulo1_wt_pifq.R`. The output folder `resultados_wt/` is created automatically by the R pipeline, so it does not need to be created manually before running the workflow. The repository also includes a lightweight Cell Ranger launcher at `tools/cellranger/cellranger`; adding that folder to `PATH` lets users type `cellranger` directly.

Docker quick start

The downstream R/Python environment is provided by `matigara/scrnaseq:latest`. The repository root is mounted as `/workspace`, so the same relative paths used on the host are visible inside R. Cell Ranger is used before Docker, from the host/server command line. The repository provides a launcher for convenience, while the licensed Cell Ranger binary itself is supplied by the server installation or by placing an official extracted Cell Ranger folder under `tools/cellranger/`.

bash
```
docker pull matigara/scrnaseq:latest

docker run -it --rm \
  -v "$PROJECT_DIR:/workspace" \
  -w /workspace \
  matigara/scrnaseq:latest /bin/bash
```

Raw data download

The raw reads for this WT/pifq test were staged from GSA accession `CRA010863`. The two runs used here were `CRR775297` and `CRR775298`. The original files were downloaded by FTP and then symlinked into the Illumina-style naming expected by Cell Ranger.

bash
```
cd "$PROJECT_DIR"
mkdir -p data/raw/CRA010863

cat > data/raw/CRA010863/CRA010863_CRR775297_298_urls.txt <<'EOF'
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775297/CRR775297_f1.fastq.gz
  out=CRR775297_f1.fastq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775297/CRR775297_r2.fastq.gz
  out=CRR775297_r2.fastq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775298/CRR775298_f1.fastq.gz
  out=CRR775298_f1.fastq.gz
ftp://download.big.ac.cn/gsa2/CRA010863/CRR775298/CRR775298_r2.fastq.gz
  out=CRR775298_r2.fastq.gz
EOF

aria2c -c -x 8 -s 8 -j 4 \
  -d data/raw/CRA010863 \
  -i data/raw/CRA010863/CRA010863_CRR775297_298_urls.txt
```

After download, the server contained four compressed FASTQ files totaling approximately 65 GB. Cell Ranger uses the `--sample` argument to select files by prefix, so the downloaded CRR files were symlinked to sample names that match the `cellranger count` commands.

bash
```
cd "$PROJECT_DIR/data/raw/CRA010863"

ln -sf CRR775297_f1.fastq.gz Scpifq_S1_L001_R1_001.fastq.gz
ln -sf CRR775297_r2.fastq.gz Scpifq_S1_L001_R2_001.fastq.gz
```

```
ln -sf CRR775298_f1.fastq.gz ScWT_S1_L001_R1_001.fastq.gz
ln -sf CRR775298_r2.fastq.gz ScWT_S1_L001_R2_001.fastq.gz
```

Sample mapping used here: `ScWT` comes from `CRR775298`, and `Scpifq` comes from `CRR775297`.

Reference preparation for Cell Ranger

Cell Ranger requires an indexed transcriptome reference generated with `cellranger mkref`. In this server run, the reference was is hosted as a raw directory on VirtualPlant and can be downloaded into the repository under `references/`. Its metadata reports `Arabi.fasta` and `Arabi.gtf` as source inputs, genome name `Ara`, Araport11 gene annotation, and `cellranger-9.0.1` as the mkref version.

bash
```
cd "$PROJECT_DIR"

cd references
wget -r -np -nH --cut-dirs=2 -R "index.html*" \
  http://virtualplant.bio.puc.cl/share/singlecell/
Arabidopsis_thaliana_Araport11_CellRanger_reference/

cd "$PROJECT_DIR"
REF="$PROJECT_DIR/references/Arabidopsis_thaliana_Araport11_CellRanger_reference"

find "$REF" -maxdepth 2 -type f | sort
cat "$REF/reference.json"
```

If the reference needs to be rebuilt from FASTA and GTF files, stage the genome FASTA and annotation GTF in a reference source folder and run `cellranger mkref`. The exact FASTA/GTF download source should be recorded with the project because it defines the gene model used for all later counts.

bash
```
mkdir -p reference_source
cd reference_source

# Cell Ranger must be available in PATH.
cellranger --version

# Place the Arabidopsis genome FASTA and Araport11 GTF here.
# Example expected local names:
#   Arabi.fasta
#   Arabi.gtf

cellranger mkref \
  --ref-version=Araport11 \
  --genome=Ara \
  --fasta=Arabi.fasta \
  --genes=Arabi.gtf \
  --nthreads=16 \
  --memgb=64
```

Cell Ranger count generation

Cell Ranger is not executed inside the R/Python Docker container. Install it once from 10x Genomics, load it from the server environment, or place the extracted official folder under `tools/cellranger/`. The repository launcher then makes the command `cellranger` available without writing a machine-specific path in the tutorial. The server run used Cell Ranger `7.1.0` for read alignment and count matrix generation. Both WT and pifq were processed with `--localcores=80` and `--no-bam`. The `--no-bam` flag reduces disk use by skipping BAM output.

bash
```
cd "$PROJECT_DIR"

REF="$PROJECT_DIR/references/Arabidopsis_thaliana_Araport11_CellRanger_reference"
FASTQS="$PROJECT_DIR/data/raw/CRA010863"

export PATH="$PROJECT_DIR/tools/cellranger:$PATH"
cellranger --version
bash workflow/run_cellranger_wt_pifq.sh
```

The helper script above keeps the Cell Ranger command short and reproducible. Internally, it verifies that `cellranger` is available, checks that the reference and FASTQs exist, writes output under `cellranger/`, and skips a sample if `outs/metrics_summary.csv` already exists.

bash
```
cellranger count \
  --localcores=80 \
  --id=Scpifq \
  --fastqs="$FASTQS" \
  --sample=Scpifq \
  --transcriptome="$REF" \
  --no-bam

cellranger count \
  --localcores=80 \
  --id=ScWT \
  --fastqs="$FASTQS" \
  --sample=ScWT \
  --transcriptome="$REF" \
  --no-bam
```

Chapter 1 uses the standard filtered matrices: `cellranger/ScWT/outs/filtered_feature_bc_matrix` and `cellranger/Scpifq/outs/filtered_feature_bc_matrix`. The Cell Ranger summaries reported 4,067 estimated WT cells and 7,006 estimated pifq cells before the downstream Seurat filtering step.

Chapter 1 in R

Bibliography marker file

Before launching Chapter 1, place a bibliography marker table in the repository root. The workflow expects a plain text, tab-separated file named `biblio_marks_custom.txt`. The file can be created in any text editor, Excel/Numbers exported as TSV, or directly in the terminal. It is not restricted to the markers shown here: any bibliography-supported marker set can be used as long as the file keeps the two required columns, `cell types` and `gene`. The row order is preserved in the marker dotplot.

Required text-file format:

bash

```
cell types      gene
Mesophyll       AT4G12970
ENDO1   AT4G34970
ENDO2   AT2G36100
Epidermis Cotyledon     AT4G21750
guard cell      AT3G24140
PARENCHYMA      AT5G23660
PHLOEM  AT1G79430
PHLOEM AND SIEVE ELEMENT        AT3G01680
XYLEM   AT4G36160
PROCAMBIUM      AT3G09070
XYLEM PARENCHYMA        AT3G58990
SAM     AT1G62360
Epidermis Hypocotyl     AT4G20260
Epidermis Hypocotyl     AT3G48970
Epidermis Hypocotyl     AT4G34769
C1      AT1G62510
C2      AT4G15920
C3      AT3G12700
```

One possible way to create that file from the terminal is:

bash

```
cd "$PROJECT_DIR"

nano biblio_marks_custom.txt
```

Save the file as tab-separated text. Repeated cell-type names are allowed when several genes support the same annotation, for example multiple `Epidermis Hypocotyl` markers.

Start the R session

Once the Cell Ranger matrices and marker file exist, start the Docker container and open R. The next blocks are meant to be copied and pasted section by section inside that R session. This keeps Chapter 1 inspectable instead of running the whole workflow as one automatic script.

bash

```
cd "$PROJECT_DIR"

docker run -it --rm \
```

```
  -v "$PROJECT_DIR:/workspace" \
  -w /workspace \
  matigara/scrnaseq:latest /bin/bash

R
```

Step 1 - Initialization

The R workflow loads the package set, plotting helpers, and core Seurat functions from the cloned repository. The output root for this dataset is `resultados_wt/`.

R

```
PIPELINE_DIR <- "/workspace/workflow"
DATA_DIR     <- "/workspace"
base_dir     <- file.path(DATA_DIR, "resultados_wt")

source(file.path(PIPELINE_DIR, "load_libraries.R"))
source(file.path(PIPELINE_DIR, "ScRNA_Analysis_Functions.R"))

set.seed(1807)
options(Seurat.allow.s4 = FALSE)
setwd(DATA_DIR)

list2env(create_pipeline_dirs(base_dir), envir = .GlobalEnv)
output_dir <- base_dir
```

Step 2 - Sample manifest

Only these two sample entries need to be changed when adapting the chapter to another pair or set of Cell Ranger matrices.

R

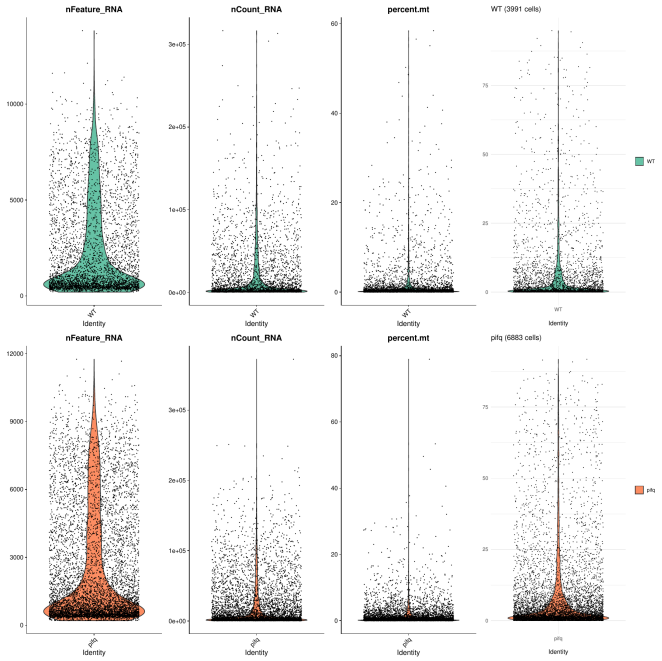
```
samples <- list(
  list(file = "cellranger/ScWT/outs/filtered_feature_bc_matrix",
       label = "WT", condition = "WT"),
  list(file = "cellranger/Scpifq/outs/filtered_feature_bc_matrix",
       label = "pifq", condition = "pifq")
)

colors <- c(WT = "#66c2a5", pifq = "#fc8d62")
mt_pattern <- "^ATMG"
cp_pattern <- "^ATCG"
```

Section 1 - Data loading and pre-filter QC

The first R section loads each Cell Ranger matrix into a Seurat object, calculates mitochondrial and chloroplast fractions using Arabidopsis gene prefixes, and saves the pre-filter QC panel. This plot is inspected before choosing filtering thresholds.

R

```
output_dir <- dir_01

seurat_list_raw <- load_seurat_samples(
  samples = samples,
  DATA_DIR = DATA_DIR,
  mt_pattern = mt_pattern,
  cp_pattern = cp_pattern
)

plot_qc_batch(seurat_list_raw, colors, "qc_prefilter.pdf")
saveRDS(seurat_list_raw, file.path(dir_objects, "seurat_list_raw.rds"))
```



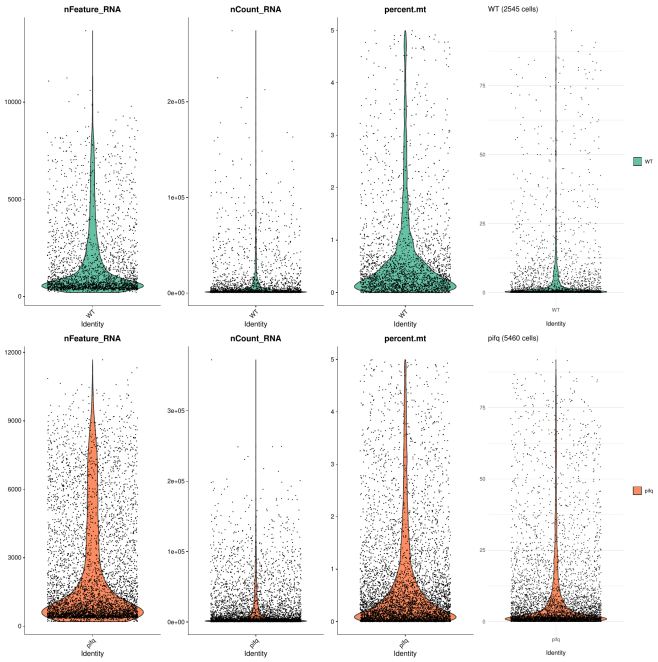
Pre-filter QC panel produced before cell filtering.

Section 2 - Cell filtering and doublet detection

The filtering step keeps cells with at least 200 detected features and less than 5% mitochondrial signal. DoubletFinder is then run sample by sample inside `filter_seurat_samples()`. The post-filter QC panel is saved after low-quality cells and putative doublets are removed.

R

```
output_dir <- dir_01

seurat_list <- filter_seurat_samples(
  seurat_list_raw,
  min_features = 200,
  max_mt = 5
)

plot_qc_batch(seurat_list, colors, "qc_postfilter.pdf")
saveRDS(seurat_list, file.path(dir_objects, "seurat_list_postfilter.rds"))
```



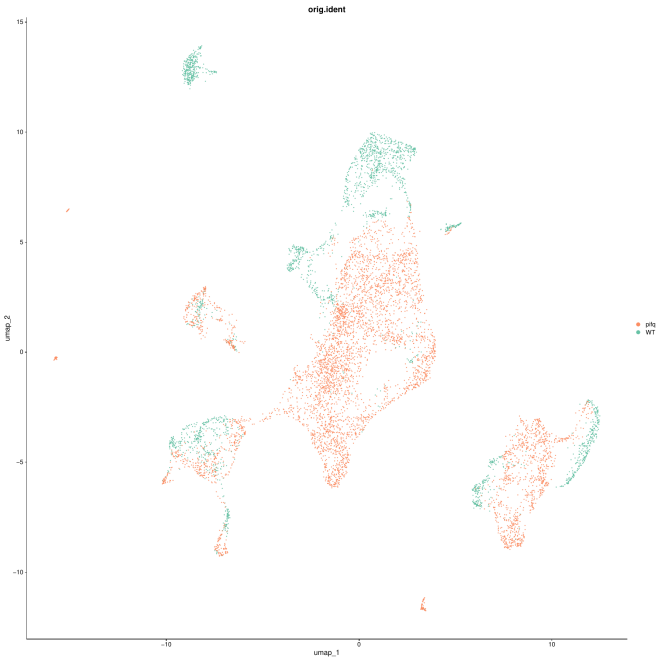
Post-filter QC panel after cell filtering and doublet removal.

Section 3 - Merge and initial preprocessing

Filtered samples are merged, normalized, scaled, reduced by PCA, and projected with UMAP. This pre-Harmony UMAP is used to inspect sample structure before integration.

R

```
output_dir <- dir_01

pbmc_harmony <- reduce(seurat_list, merge) |>
  NormalizeData(verbose = FALSE) |>
  FindVariableFeatures(selection.method = "vst", nfeatures = 2000,
                       verbose = FALSE) |>
  ScaleData(verbose = FALSE) |>
  RunPCA(npcs = 30, verbose = FALSE) |>
  RunUMAP(reduction = "pca", dims = 1:30, verbose = FALSE)

save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
         "umap_preharmony.pdf")
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_preharmony.rds"))
```



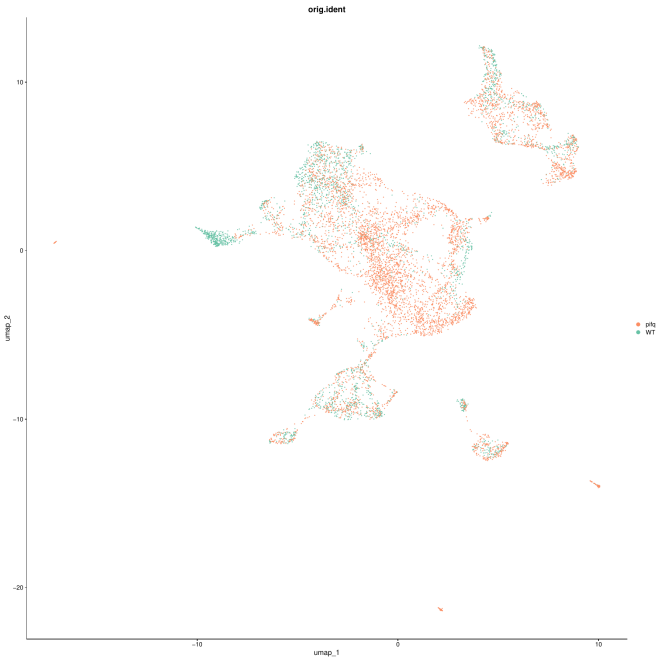
UMAP before Harmony integration, colored by sample.

Section 4 - Harmony batch correction

Harmony is applied using `orig.ident` as the integration variable, and a second UMAP is generated from the Harmony reduction.

R

```
output_dir <- dir_01

pbmc_harmony <- pbmc_harmony |>
  RunHarmony("orig.ident", plot_convergence = FALSE) |>
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE)

save_pdf(DimPlot(pbmc_harmony, group.by = "orig.ident", cols = colors),
         "umap_postharmony.pdf")
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_postharmony.rds"))
```



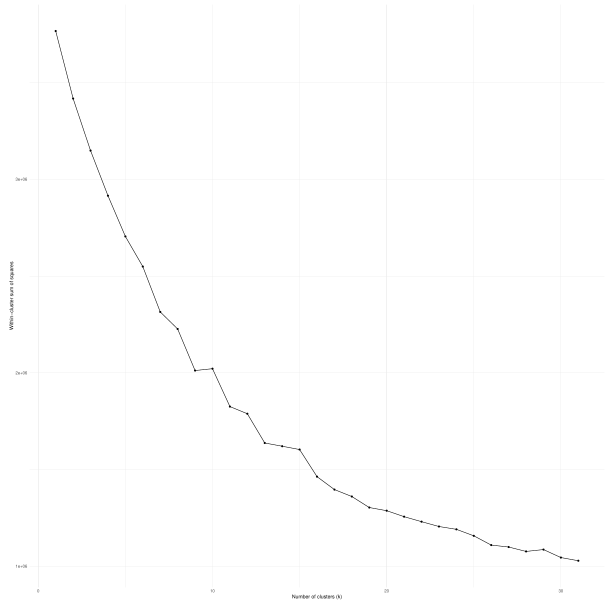
UMAP after Harmony integration, colored by sample.

Section 5 - Resolution optimization

The resolution optimization section first generates a small elbow plot and then runs the clustree resolution sweep. These outputs are inspected before choosing the final clustering resolution.

R

```
resolutions_test <- c(0.15, 0.35, 0.45, 0.55, 1.0)
output_dir <- dir_02

k_range <- 1:31
pca_data <- Embeddings(pbmc_harmony, "pca")[, 1:30]
wss <- sapply(k_range, function(k) {
  kmeans(pca_data, centers = k, nstart = 4)$tot.withinss
})

elbow_plot <- ggplot(data.frame(k = k_range, wss = wss), aes(k, wss)) +
  geom_line() +
  geom_point() +
  labs(x = "Number of clusters (k)",
       y = "Within-cluster sum of squares") +
  theme_minimal()

save_pdf(elbow_plot, "elbow_plot.pdf", w = 18, h = 18)
```

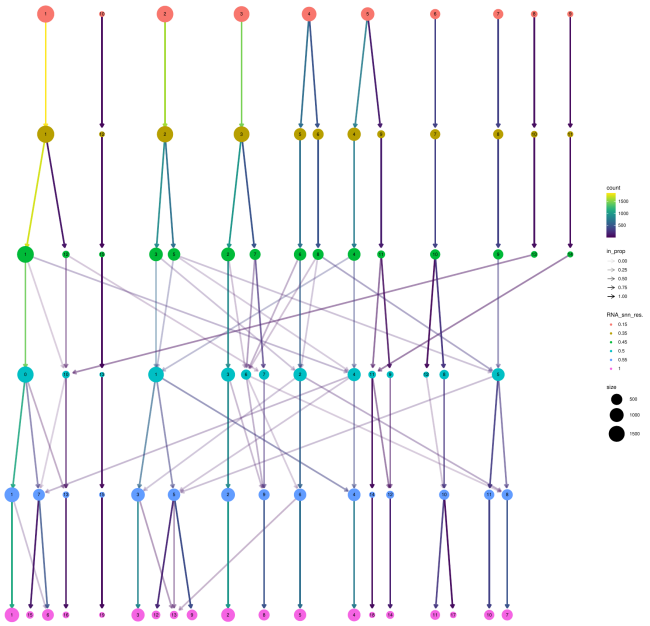


Elbow plot generated during resolution optimization.

R

```
clu <- pbmc_harmony |>
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) |>
  FindNeighbors(reduction = "harmony", dims = 1:30,
                k.param = 20, verbose = FALSE)

for (res in resolutions_test) {
  clu <- FindClusters(clu, resolution = res,
```

```
                        algorithm = 4, verbose = FALSE)
}

save_pdf(clustree(clu, prefix = "RNA_snn_res."),
         "clustree2.pdf", w = 18, h = 18)
saveRDS(clu, file.path(dir_objects, "resolution_sweep_clu.rds"))
```



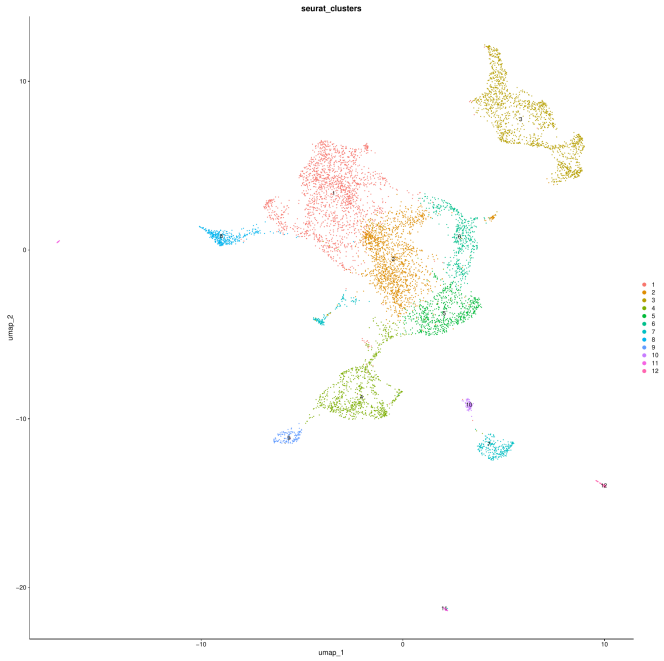
Clustree resolution sweep.

Section 6 - Final clustering

The final Leiden resolution is set after inspecting the clustree output. A cluster-labeled UMAP is saved as the first final clustering diagnostic.

R

```
cluster_resolution <- 0.35
output_dir <- dir_02

pbmc_harmony <- pbmc_harmony |>
  RunUMAP(reduction = "harmony", dims = 1:30, verbose = FALSE) |>
  FindNeighbors(reduction = "harmony", dims = 1:30,
                k.param = 20, verbose = FALSE) |>
  FindClusters(resolution = cluster_resolution,
               algorithm = 4, verbose = FALSE)

Idents(pbmc_harmony) <- "seurat_clusters"
save_pdf(DimPlot(pbmc_harmony, group.by = "seurat_clusters", label = TRUE),
         "umap_seuratclusters.pdf")
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_clusters.rds"))
```



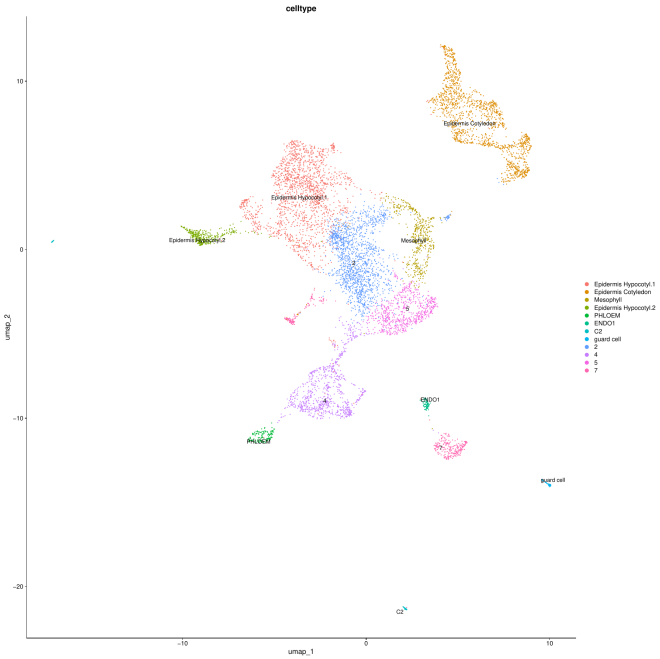
Final Leiden clusters at resolution 0.35.

Section 7 - Bibliography marker annotation

Annotation uses a marker table with two columns: `cell types` and `gene`. Marker order in this file controls the Y-axis order of the dotplot, and the X-axis follows the matching cell-type order. Numeric suffixes such as `Epidermis Hypocotyl.1` and `Epidermis Hypocotyl.2` remain adjacent to their parent marker category.

R
```
biblio_marks_file <- file.path(DATA_DIR, "biblio_marks_custom.txt")
marker_table <- read.table(biblio_marks_file, header = TRUE,
                           sep = "\t", quote = "")

markers <- find_markers(
  pbmc_harmony,
  output_file = file.path(output_dir, "FindAllMarkers.tsv")
)

pbmc_harmony <- annotate_by_markers(
    pbmc_harmony,
  markers,
  reference_file = biblio_marks_file
)
```

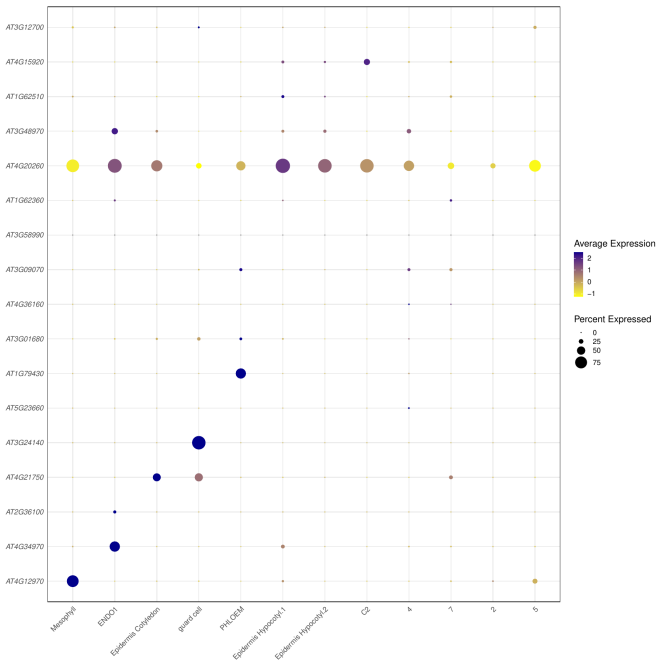
R
```
save_pdf(
  DimPlot(pbmc_harmony, group.by = "celltype",
          label = TRUE, repel = TRUE, raster = FALSE),
  "umap_annotation_biblio.pdf"
)
saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_annotated.rds"))
```



UMAP labeled by bibliography marker annotation.

R

```
plot_marker_dotplot(
  pbmc_harmony,
  marker_table,
  annot_col = "celltype",
  outfile = file.path(output_dir,
                      "dotplot_marker_table_annotation_biblio.pdf"),
  width = 18,
  height = 18
)
```



Marker dotplot ordered by the bibliography marker file.

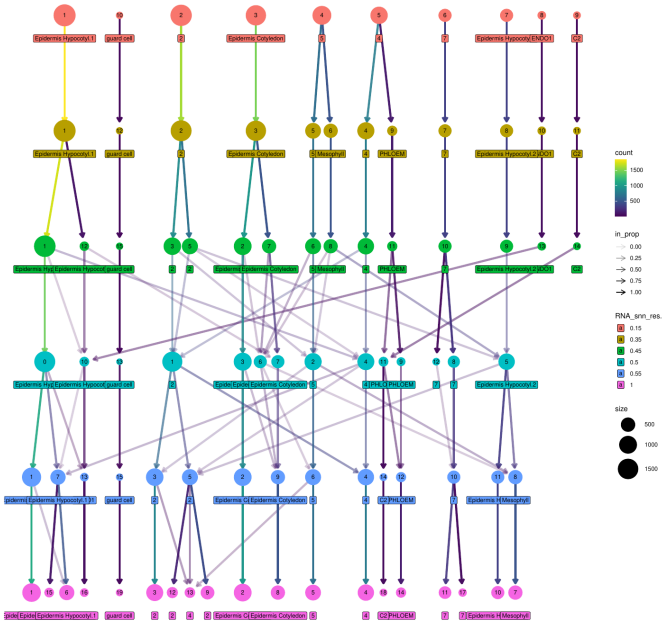
Numeric labels were intentionally retained when no bibliography marker match was detected for that cluster.

Section 8 - Annotated clustree

This block reuses the resolution sweep object from Section 5. It does not recalculate neighbors or clusters. The final bibliography-based annotation is copied into the sweep object and summarized on each node by the most frequent cell type.

R

```
output_dir <- dir_03

Mode <- function(x) {
  x <- as.character(x)
  x <- x[!is.na(x) & nzchar(x)]
  if (length(x) == 0) return(NA_character_)
  names(sort(table(x), decreasing = TRUE))[1]
}

stopifnot(exists("clu"))
stopifnot("celltype" %in% colnames(pbmc_harmony@meta.data))

celltype_label <- as.character(pbmc_harmony$celltype)
names(celltype_label) <- Cells(pbmc_harmony)
clu$celltype_label <- celltype_label[Cells(clu)]

print(table(clu$celltype_label, useNA = "ifany"))

save_pdf(
  clustree(clu, prefix = "RNA_snn_res.",
           node_label = "celltype_label", node_label_aggr = "Mode"),
  "clustree_annotated.pdf", w = 14, h = 14
)
```



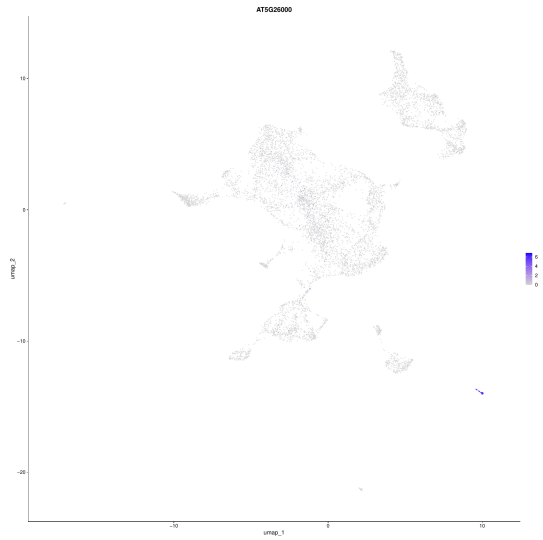
Clustree nodes labeled with the dominant bibliography annotation.

Section 9 - Gene expression visualization

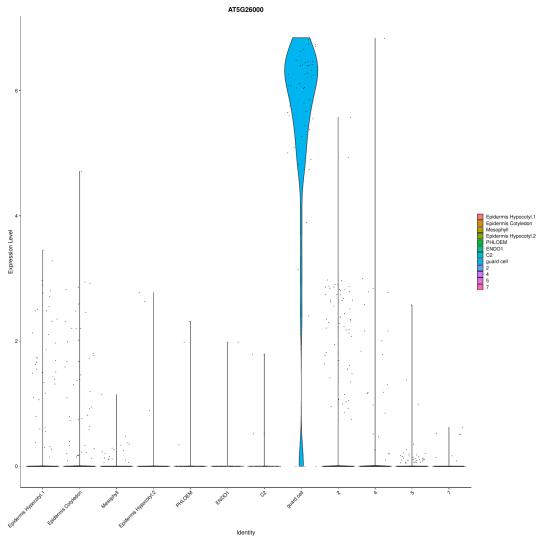
The expression block is intentionally simple: one gene, one global UMAP FeaturePlot, and one global violin plot. No cell-type-specific zoom or two-gene comparison is included by default.

R

```
gene <- "AT5G26000"
output_dir <- dir_04

pbmc_harmony <- JoinLayers(pbmc_harmony)
Idents(pbmc_harmony) <- "celltype"

save_pdf(FeaturePlot(pbmc_harmony, features = gene),
         "feature_gene_all.pdf")
save_vln(VlnPlot(pbmc_harmony, features = gene),
         "vln_gene_all.pdf")
```



Global FeaturePlot for AT5G26000.



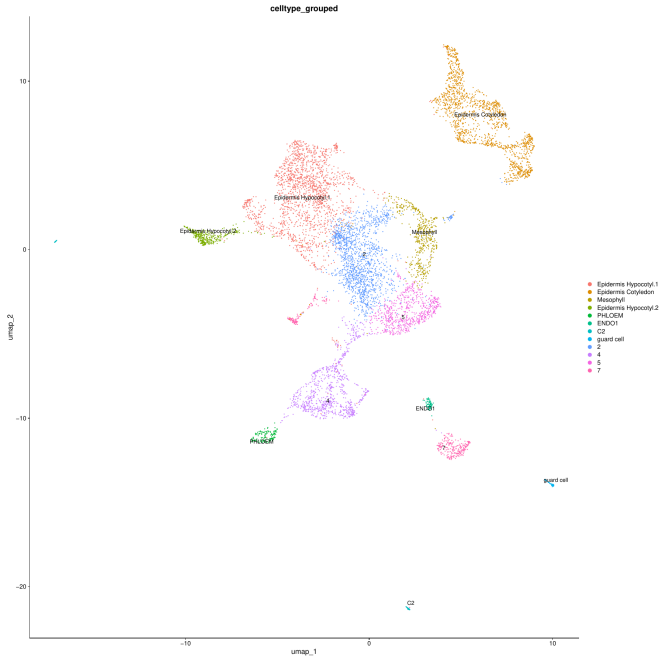
Global violin plot for AT5G26000.

Section 10 - Cell-type grouping [optional]

Fine-grained labels can be collapsed into broader categories for later analyses. For the current WT/pifq test, no grouping is applied, so `celltype_grouped` is a direct copy of `celltype`. The executed default remains empty. The commented example shows how to merge two Hypocotyl epidermis labels into one broader label if that is needed later.

R

```
# Example only. Leave commented unless you want to merge these labels:
# grouping <- c(
#   "Epidermis Hypocotyl.1" = "Epidermis Hypocotyl",
#   "Epidermis Hypocotyl.2" = "Epidermis Hypocotyl"
# )

grouping <- c()
output_dir <- dir_05

if (length(grouping) > 0) {
  pbmc_harmony$celltype_grouped <- recode(pbmc_harmony$celltype, !!!grouping)
} else {
  pbmc_harmony$celltype_grouped <- pbmc_harmony$celltype
}

save_pdf(
  DimPlot(pbmc_harmony, group.by = "celltype_grouped",
          label = TRUE, repel = TRUE, raster = FALSE),
  "umap_grouped.pdf"
)
```



Grouping output with no labels merged in the current run.

Section 11 - Interactive cell-type inspection [optional]

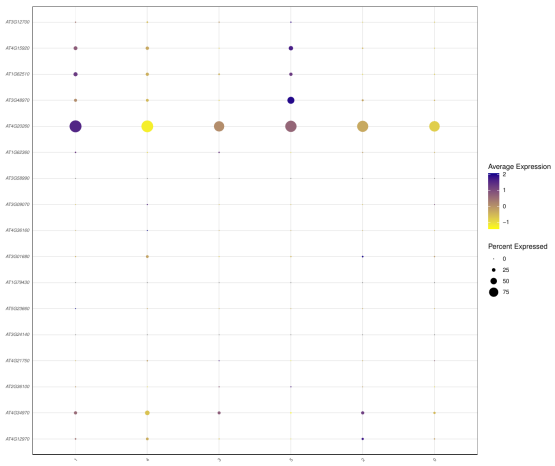
This section is intentionally manual. The user chooses one annotated cell type, runs the inspection, opens the figures, and then decides whether any reassignment is biologically justified. The example below was tested only for `Epidermis Hypocotyl.1`. It does not modify `pbmc_harmony` and does not create a curated annotation by itself.

R

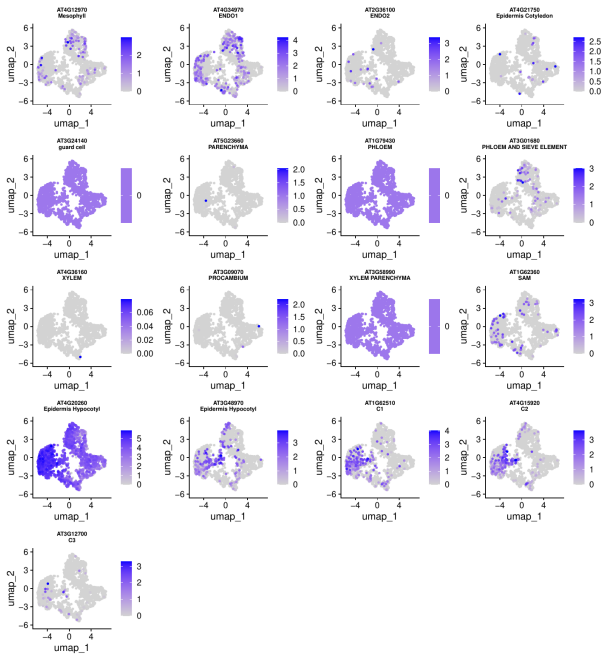
```
output_dir <- dir_05

print(table(pbmc_harmony$celltype, useNA = "ifany"))

epi_hypo1_inspection <- inspect_subcluster_markers(
  pbmc_harmony,
  tipo = "Epidermis Hypocotyl.1",
  marker_table = marker_table,
  output_dir = output_dir,
  annot_col = "celltype",
  resolution = 0.3,
  dims = 1:20,
  prefix = "Epidermis_Hypocotyl_1"
)

epi_hypo1 <- epi_hypo1_inspection$object
print(table(epi_hypo1$cluster_subtipo, useNA = "ifany"))
epi_hypo1_inspection$files
```

The inspection produces three files for review: the original UMAP coordinates subsetted to the inspected cells and colored by the new subclusters, a dotplot using all bibliography markers available in the object, and a small FeaturePlot panel for those markers.

Original UMAP subset colored by Epidermis Hypocotyl.1 subclusters.



Bibliography-marker dotplot across Epidermis Hypocotyl.1 subclusters.



Small FeaturePlot panel for bibliography markers in Epidermis Hypocotyl.1.

Section 12 - Export to h5ad

The final object can be exported for Python workflows. If manual curation was skipped, `celltype_grouped` is copied to `celltype_curated` before export.

R

```
if (!"celltype_curated" %in% colnames(pbmc_harmony@meta.data)) {
  message("celltype_curated not found; using celltype_grouped for export.")
  pbmc_harmony$celltype_curated <- pbmc_harmony$celltype_grouped
  saveRDS(pbmc_harmony, file.path(dir_objects, "pbmc_harmony_curated.rds"))
}

export_to_scanpy(
  pbmc_harmony,
  file.path(dir_objects, "pbmc_harmony_curated.h5ad")
)
```

The Chapter 1 run finished with 23,657 genes and 8,005 cells after filtering and doublet removal: 2,545 WT cells and 5,460 pifq cells. The main exported files are `resultados_wt/objects/pbmc_harmony_curated.rds`, `resultados_wt/objects/pbmc_harmony_curated.h5ad`, `resultados_wt/03_annotation/FindAllMarkers.tsv`, and `resultados_wt/sessionInfo.txt`.

Software versions

The downstream R and Python environment comes from `matigara/scrnaseq:latest`. Cell Ranger is not installed inside this Docker image; it was run externally on the server. Full TSV exports are stored with the report assets, while this appendix keeps only the versions needed to reproduce Chapter 1 without filling the PDF with long tables.

Command-line programs

`R` 4.5.3 (2026-03-11) -- "Reassured Reassurer"; `python3` Python 3.12.3; `pip` pip 26.1.2 from /opt/venv/lib/python3.12/site-packages/pip (python 3.12); `pandoc` pandoc 3.1.3. Cell Ranger `7.1.0` was used for `count`, and Cell Ranger `9.0.1` was recorded in the hosted mkref reference.

Key R packages in Docker

- `Seurat` 5.5.0
- `SeuratObject` 5.4.0
- `DoubletFinder` 2.0.6
- `harmony` 2.0.3
- `clustree` 0.5.1
- `ggplot2` 4.0.3
- `dplyr` 1.2.1
- `Matrix` 1.7-5
- `SingleCellExperiment` 1.32.0
- `zellkonverter` 1.20.1
- `anndataR` 1.0.2

Key Python packages in Docker

- `scanpy` 1.12.1
- `anndata` 0.12.16
- `scFates` 1.2.4
- `palantir` 1.4.4
- `cellbender` 0.3.2
- `numpy` 2.4.6
- `pandas` 2.3.3
- `scipy` 1.17.1
- `h5py` 3.16.0
- `matplotlib` 3.10.9
- `seaborn` 0.13.2
- `leidenalg` 0.12.0
- `python-igraph` 1.0.0